Employing point-in-time lookups fixes this problem by handling the time matching for you. For those familiar with *The Price Is Right*, it's the closest feature value without going over. More specifically, it will match with the latest feature value as of the event timestamp without ever providing a feature value calculated after the event. You don't worry about data leakage during training. Without a `Timeseries` primary key, the latest value for a feature is matched.

> **Note**
>
> It's recommended to apply Z-ordering on time series tables for better performance in point-in-time lookups. Z-ordering is covered in *Chapter 3*.

Looking up features is great and often the best choice for your use case. However, in some data science and machine learning tasks, we need feature values calculated quickly on data that we do not have ahead of time. For those projects, we need on-demand features.

Computing on-demand features

Calculating the number of transactions per customer in a brief time window works in a streaming fashion because we only need to use historical data. When we want to use a feature that requires data available only at inference time, we use on-demand features, with unknown values until inference time. In Databricks, you can create on-demand features with Python **user-defined functions** (**UDFs**). These Python UDFs can then be invoked via `training_set` configurations to create training datasets, as you will see in *Chapter 6*.

Let's consider the *Streaming Transactions* project again. We want to add a feature for the amount a product sold at, compared to its historical maximum price, and use this as part of the training data to predict the generated classification label. In this scenario, we don't know the purchase price until the transaction has been received. We'll cover how to build a Python UDF for calculating an on-demand feature for the *Streaming Transactions* project in the *Applying our learning* section.

Additionally, we recommend checking out the *How Databricks AI improves model accuracy with real-time computations* and *Best Practices for Realtime Feature Computation on Databricks* articles for in-depth advice from the on-demand experts at Databricks; see the *Further reading* section for the links.

We have discussed saving features to feature tables in Unity Catalog, the standard "offline" pattern for applications that do not have low-latency requirements. If your business problem requires low latency or fast results, you'll need your data in an online table or implement Databricks Feature Serving. Feature Serving can serve functions as well as precomputed features. For low-latency projects, we recommend Databricks Model Serving because it removes any need for Feature Serving. We don't cover Feature Serving in this book, but if you are going to serve your models externally to Databricks, Feature Serving may be of interest to you.